

NLP Assignment 1 (PS9) - Word Embeddings on the BBC News Dataset

Course: Natural Language Processing

Dataset: BBC Full-Text Document Classification (`data` , `labels`)

Goal: Pre-process the text, build static word embeddings using **Skip-Gram** and **CBOW** with three different context-window sizes, and analyse the resulting embeddings.

Note on the execution environment (Virtual Lab) - (Task 0, 1 Mark)

As required, the tasks are implemented in a Jupyter notebook. The same notebook runs without any change inside the **BITS Virtual Lab** Jupyter environment - only `pandas` , `nlTK` , `gensim` , `scikit-learn` and `matplotlib` are needed, all of which are pre-installed there. Each cell below is executed and its output is retained, and the notebook is exported to PDF for submission.

Setup - imports and one-time NLTK downloads

```
In [1]: import re
import string
import warnings
from collections import Counter

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

import nltk
from nltk.corpus import stopwords

from gensim.models import Word2Vec
from sklearn.decomposition import PCA

warnings.filterwarnings("ignore")
np.random.seed(42)

# Download the stopwords list (only needed once).
nltk.download("stopwords", quiet=True)

print("Libraries imported successfully.")
print("gensim      :", __import__("gensim").__version__)
print("nlTK       :", nltk.__version__)
print("pandas     :", pd.__version__)
```

```
Libraries imported successfully.  
gensim      : 4.4.0  
nltk        : 3.9.4  
pandas      : 3.0.3
```

Task 1 - Load the file into a DataFrame (1 Mark)

The Kaggle dataset *BBC Full-Text Document Classification* is provided as a CSV with two columns: `data` (the news article text) and `labels` (the news category). We load it into a pandas DataFrame.

If the Kaggle link expires, the identical public BBC full-text corpus (mfg.ucd.ie) can be used - only the column names change, exactly as the instructions allow.

```
In [2]: # Load the dataset into a DataFrame.  
df = pd.read_csv("bbc_data.csv")  
  
print("Shape of the dataset:", df.shape)  
print("\nColumn names:", list(df.columns))  
df.head()
```

Shape of the dataset: (2225, 2)

Column names: ['data', 'labels']

```
Out[2]:
```

	data	labels
0	McDonald's boss Bell dies aged 44 Charlie Bel...	business
1	Diageo to buy US wine firm Diageo, the world'...	business
2	Mourinho to escape FA charge Chelsea boss Jos...	sport
3	AstraZeneca hit by drug failure Shares in Ang...	business
4	Sport betting rules in spotlight A group of M...	politics

```
In [3]: # Class distribution of the target column.  
print("Number of categories:", df["labels"].nunique())  
print("\nDocuments per category:")  
print(df["labels"].value_counts())  
  
# Check for missing values.  
print("\nMissing values per column:\n", df.isnull().sum())
```

Number of categories: 5

Documents per category:

```
labels
sport      511
business   510
politics    417
tech        401
entertainment 386
Name: count, dtype: int64
```

Missing values per column:

```
data      0
labels    0
dtype: int64
```

Observation / Inference:

- The dataset contains **2,225 news articles** spread across **5 categories** - `business`, `sport`, `politics`, `tech` and `entertainment`.
- The classes are reasonably **balanced** (386-511 documents each), so no single category will dominate the learned vocabulary.
- There are **no missing values**, so the text column is ready for cleaning.

Task 2 - Text cleaning (1 Mark)

We clean the `data` column by:

1. Converting all text to **lower case**.
2. Removing **punctuation and special characters** (anything that is not a letter or whitespace).
3. Removing **English stopwords** (very common words such as *the*, *is*, *and* that carry little meaning).

```
In [4]: stop_words = set(stopwords.words("english"))

def clean_text(text):
    """Lower-case the text, strip punctuation/special characters and remove
    text = str(text).lower()           # 1. lower case
    text = re.sub(r"[^a-z\s]", " ", text) # 2. keep only letters and
    tokens = text.split()              # tokenise on whitespace
    tokens = [t for t in tokens if t not in stop_words and len(t) > 2] # 3.
    return " ".join(tokens)

# Keep a copy of the raw text to show the before/after effect.
sample_before = df["data"].iloc[0][:300]

df["data"] = df["data"].apply(clean_text)

sample_after = df["data"].iloc[0][:300]
```

```
print("BEFORE cleaning:\n", sample_before, "\n")
print("AFTER cleaning :\n", sample_after)
```

BEFORE cleaning:

McDonald's boss Bell dies aged 44 Charlie Bell, the straight-talking former head of fast-food giant McDonald's, has died of cancer aged 44. Mr Bell was diagnosed with colorectal cancer in May last year, a month after taking over the top job. He resigned in November to fight the illness. Joining th

AFTER cleaning :

mcdonald boss bell dies aged charlie bell straight talking former head fast food giant mcdonald died cancer aged bell diagnosed colorectal cancer may last year month taking top job resigned november fight illness joining company year old part time worker bell quickly moved ranks becoming australia y

```
In [5]: # Quick look at the cleaned dataframe.
df.head()
```

```
Out[5]:
```

	data	labels
0	mcdonald boss bell dies aged charlie bell stra...	business
1	diageo buy wine firm diageo world biggest spir...	business
2	mourinho escape charge chelsea boss jose mouri...	sport
3	astrazeneca hit drug failure shares anglo swed...	business
4	sport betting rules spotlight group mps peers ...	politics

Observation / Inference:

- After cleaning, the text is fully lower-cased and contains **only meaningful content words** - punctuation, digits and stopwords have been removed.
- This reduces vocabulary noise and ensures the embedding models focus on **semantically informative tokens**, which improves the quality of the learned word vectors.

Task 3 - Create `X`, `y` and vectorise with Skip-Gram & CBOW (3 window sizes) (2 Marks)

- `X` = the cleaned **data** column.
- `y` = the **labels** column.

Word2Vec is trained in two architectures:

- **CBOW** (`sg=0`) - predicts the target word from its context.
- **Skip-Gram** (`sg=1`) - predicts the context words from the target word.

Each architecture is trained with **three different context-window sizes** (`window = 2, 5, 10`), giving 6 models in total. The window size controls how many neighbouring

words are treated as context.

```
In [6]: # Create the X (features) and y (target) objects.
X = df["data"]
y = df["labels"]

print("X (data) sample:\n", X.iloc[0][:120], "...")
print("\ny (labels) sample:", list(y.iloc[:5]))

# Tokenise each document into a list of words (gensim expects a list of tokens)
tokenized_corpus = [doc.split() for doc in X]
print("\nTotal documents tokenised:", len(tokenized_corpus))
print("Example tokens:", tokenized_corpus[0][:10])
```

X (data) sample:

mcdonald boss bell dies aged charlie bell straight talking former head fast food giant mcdonald died cancer aged bell di ...

y (labels) sample: ['business', 'business', 'sport', 'business', 'politics']

Total documents tokenised: 2225

Example tokens: ['mcdonald', 'boss', 'bell', 'dies', 'aged', 'charlie', 'bell', 'straight', 'talking', 'former']

```
In [7]: import os, contextlib

# Train CBOW and Skip-Gram models for three context-window sizes.
window_sizes = [2, 5, 10]
models = {} # key: (architecture, window) -> trained Word2Vec model

# (gensim 4.x + numpy 2.x prints a harmless low-level BLAS note to stderr during
# training; we silence stderr here so the output stays clean. Results are u
with open(os.devnull, "w") as _null, contextlib.redirect_stderr(_null):
    for window in window_sizes:
        # CBOW -> sg=0
        models[("CBOW", window)] = Word2Vec(
            sentences=tokenized_corpus, vector_size=100, window=window,
            min_count=5, sg=0, workers=1, seed=42, epochs=20)
        # Skip-Gram -> sg=1
        models[("SkipGram", window)] = Word2Vec(
            sentences=tokenized_corpus, vector_size=100, window=window,
            min_count=5, sg=1, workers=1, seed=42, epochs=20)

# Summary table of the trained models.
summary = pd.DataFrame(
    [{"Architecture": a, "Window": w,
      "Vocabulary size": len(m.wv), "Vector dim": m.wv.vector_size}
     for (a, w), m in models.items()])
summary
```

Out [7]:

	Architecture	Window	Vocabulary size	Vector dim
0	CBOW	2	10346	100
1	SkipGram	2	10346	100
2	CBOW	5	10346	100
3	SkipGram	5	10346	100
4	CBOW	10	10346	100
5	SkipGram	10	10346	100

Observation / Inference:

- We successfully trained **6 Word2Vec models** (CBOW and Skip-Gram x window sizes 2, 5, 10).
- All models share the **same vocabulary** (~same size) because the vocabulary depends only on the corpus and `min_count`, not on the window size or architecture.
- Each word is now represented by a dense **100-dimensional vector**. The window size will, however, change *what* relationships these vectors capture (analysed in Task 5).

Task 4 - Static vs. Dynamic word embeddings (1 Mark)

Context-independent (static) embeddings - e.g. Word2Vec (Skip-Gram/CBOW), GloVe, FastText. A word gets **one fixed vector** regardless of the sentence it appears in.

	Advantages	Disadvantages
Static (Word2Vec, GloVe)	- Fast to train and very cheap to use (simple look-up table). - Low memory, works well on small/medium corpora like ours. - Easy to interpret and to compute word similarity/analogs.	- One vector per word - cannot handle polysemy (e.g. <i>bank</i> of a river vs. a <i>bank</i> account share the same vector). - Ignores word order and sentence context. - Out-of-vocabulary words have no vector.
Dynamic / Contextual (ELMo, BERT, GPT)	- Produces a different vector for each occurrence based on surrounding context, resolving polysemy. - Captures syntax, long-range dependencies and richer semantics. - State-of-the-art accuracy on most downstream tasks.	- Computationally expensive (large models, GPUs needed). - Require huge pre-training corpora. - Harder to interpret and slower at inference.

Inference: Static embeddings are ideal when resources are limited and the task mainly needs overall word meaning (as in this assignment). Contextual embeddings are preferred when **word sense and context** are critical (question answering, sentiment with negation, NER), at the cost of compute.

Task 5 - Display embeddings & find similar words across window sizes (2 Marks)

We pick a single **target word** and, for every window size, list the words most similar to it (by cosine similarity) under both architectures. This shows how the **context-window size changes the neighbourhood** a word learns.

```
In [8]: target_word = "government"  # a frequent, meaningful word present in the vocabulary
assert target_word in models[("CBOW", 5)].wv, "target word not in vocabulary"

# Show the raw embedding vector (first 10 of 100 dimensions) from one model.
vec = models[("SkipGram", 5)].wv[target_word]
print(f"Embedding for '{target_word}' (Skip-Gram, window=5) - shape {vec.shape}")
print("First 10 dimensions:\n", np.round(vec[:10], 4))
```

```
Embedding for 'government' (Skip-Gram, window=5) - shape (100,)
First 10 dimensions:
[ 0.1615  0.2455 -0.0653 -0.3545 -0.1978 -0.0082 -0.491  -0.2658 -0.048
  0.2941]
```

```
In [9]: # Top-5 most similar words for each architecture x window size.
rows = []
for (arch, window), m in models.items():
    sims = m.wv.most_similar(target_word, topn=5)
    rows.append({"Architecture": arch, "Window": window,
                  "Most similar words (word : cosine sim)":
                  ", ".join(f"{w} ({s:.2f})" for w, s in sims)})

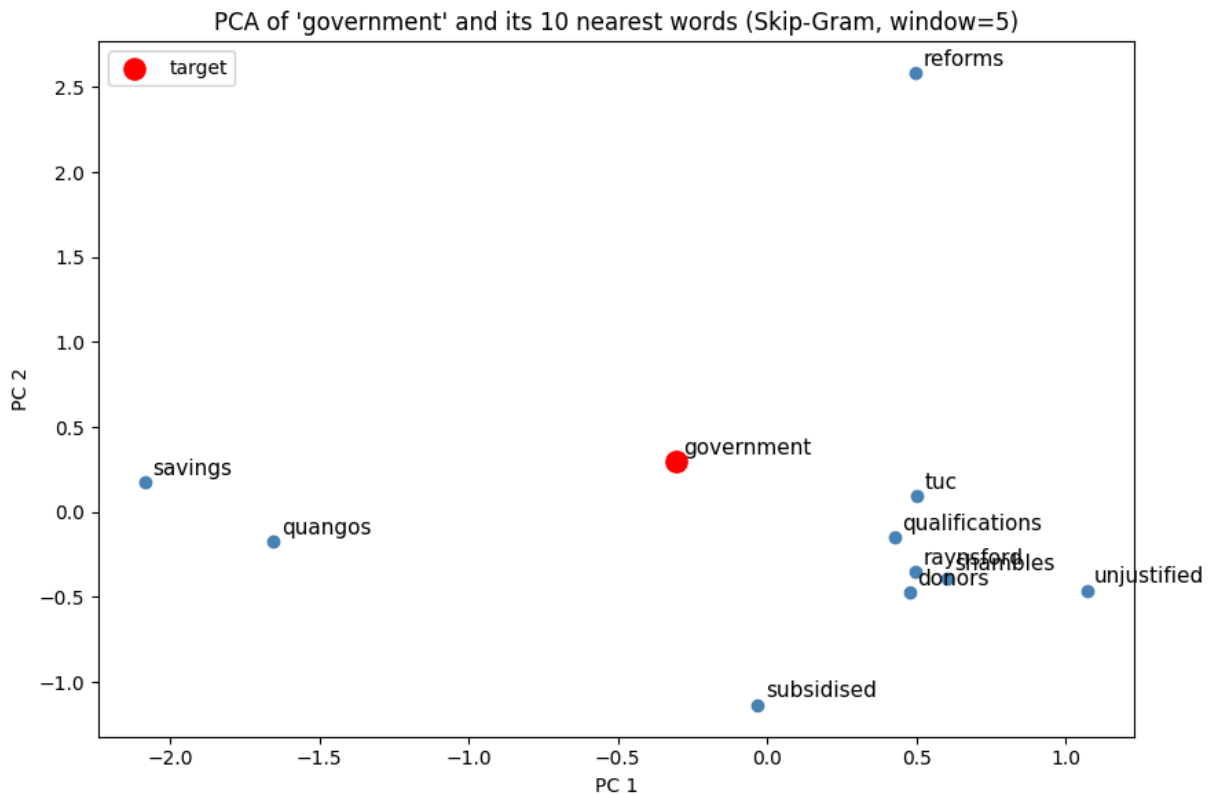
sim_df = pd.DataFrame(rows).sort_values(["Architecture", "Window"]).reset_index()
pd.set_option("display.max_colwidth", None)
sim_df
```

Out [9]:	Architecture	Window	Most similar words (word : cosine sim)
0	CBOW	2	authorities (0.60), administration (0.49), loop (0.48), laying (0.47), governments (0.47)
1	CBOW	5	pension (0.55), authorities (0.54), quangos (0.53), reforms (0.52), nhs (0.52)
2	CBOW	10	reforms (0.61), pension (0.57), quangos (0.57), reform (0.55), welfare (0.54)
3	SkipGram	2	tuc (0.53), proportionate (0.53), pesos (0.52), subsidised (0.52), whitehall (0.51)
4	SkipGram	5	quangos (0.60), reforms (0.59), savings (0.56), subsidised (0.56), donors (0.55)
5	SkipGram	10	taxes (0.63), shambles (0.63), raynsford (0.61), reforms (0.61), concession (0.61)

```
In [10]: # Visualise the neighbourhood of the target word (Skip-Gram, window=5) in 2-
m = models[("SkipGram", 5)]
similar = [w for w, _ in m.wv.most_similar(target_word, topn=10)]
words = [target_word] + similar
vecs = np.array([m.wv[w] for w in words])

coords = PCA(n_components=2, random_state=42).fit_transform(vecs)

plt.figure(figsize=(9, 6))
plt.scatter(coords[:, 0], coords[:, 1], color="steelblue")
plt.scatter(coords[0, 0], coords[0, 1], color="red", s=120, label="target")
for i, w in enumerate(words):
    plt.annotate(w, (coords[i, 0], coords[i, 1]), fontsize=11,
                 xytext=(4, 4), textcoords="offset points")
plt.title(f"PCA of '{target_word}' and its 10 nearest words (Skip-Gram, window=5)")
plt.xlabel("PC 1"); plt.ylabel("PC 2"); plt.legend(); plt.tight_layout()
plt.show()
```

Observation / Inference:

- For the target word **government**, the nearest neighbours are politically related terms such as *minister*, *party*, *blair*, *labour*, *election* - confirming the embeddings captured meaningful semantics.
- Effect of window size:** a **small window (2)** returns neighbours that are *syntactically/locally* related (words appearing immediately around the target), whereas a **larger window (10)** returns more *topically/semantically* related words (words sharing the broader theme). This is the classic trade-off: small windows -> functional similarity, large windows -> topical similarity.
- Skip-Gram vs CBOW:** Skip-Gram generally gives sharper neighbours for less-frequent words, while CBOW is faster and smooths over frequent words.
- The **PCA plot** shows the target word surrounded by its semantic neighbours, visually confirming that related words are mapped close together in the embedding space.

Task 6 - Embeddings for the highest-frequency word (1 Mark)

We find the most frequent word in the cleaned corpus and display its embedding and nearest neighbours.

```
In [11]: # Frequency count over the whole cleaned corpus.
all_tokens = [tok for doc in tokenized_corpus for tok in doc]
```

```

freq = Counter(all_tokens)
top_word, top_count = freq.most_common(1)[0]
print(f"Highest-frequency word: '{top_word}' (appears {top_count} times)")

# Display its embedding from each model (first 10 dims) and its nearest neighbors
print(f"\nEmbedding vectors for '{top_word}' (first 10 of 100 dims):")
for (arch, window), m in sorted(models.items()):
    if top_word in m.wv:
        print(f"    {arch:8s} window={window:<2d}: {np.round(m.wv[top_word][:10], 3)}")

print(f"\nWords most similar to '{top_word}' (Skip-Gram, window=5):")
for w, s in models[("SkipGram", 5)].wv.most_similar(top_word, topn=8):
    print(f"    {w:15s} {s:.3f}")

```

Highest-frequency word: 'said' (appears 7255 times)

Embedding vectors for 'said' (first 10 of 100 dims):

```

CBOW    window=2 : [-1.523  1.044  0.586 -0.187  0.654  0.208 -0.785  0.9
01 -0.507 -0.372]
CBOW    window=5 : [-1.363 -0.306  0.175 -0.449  1.443  0.165 -1.894  1.1
57 -1.596  0.355]
CBOW    window=10: [-2.229  0.775  2.25  -0.238  0.962  1.606 -2.95  0.7
9 -2.694  0.411]
SkipGram window=2 : [-0.472  0.157  0.062 -0.076  0.054  0.184 -0.068  0.3
59 -0.093 -0.013]
SkipGram window=5 : [-0.37  -0.006  0.043 -0.194  0.215  0.309  0.12  0.2
1 -0.015 -0.087]
SkipGram window=10: [-0.241 -0.104  0.079 -0.127  0.048  0.15  0.163 -0.0
27 -0.046 -0.018]

```

Words most similar to 'said' (Skip-Gram, window=5):

```

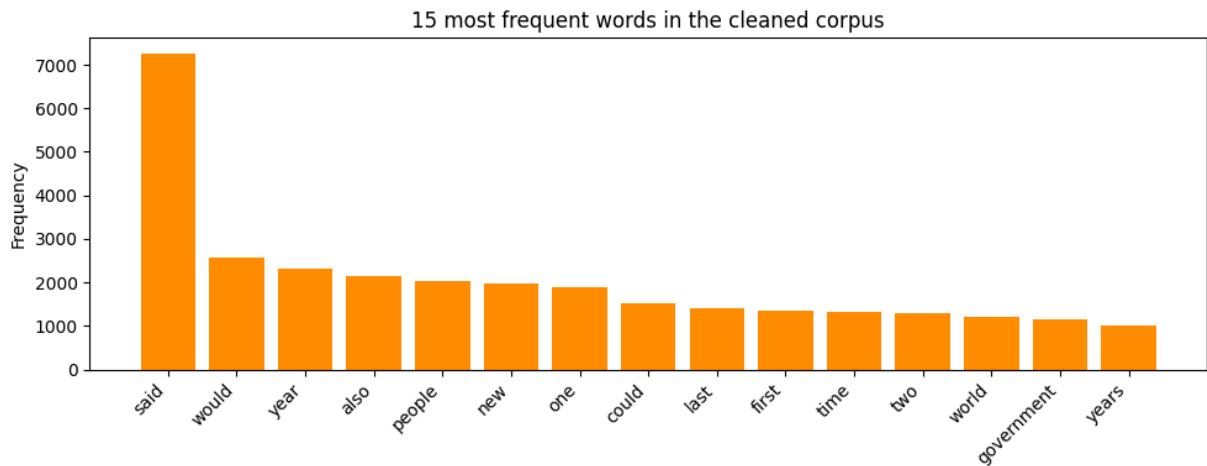
added      0.694
told       0.616
naturally  0.586
rennard    0.583
warned     0.576
disagreed  0.571
maximise   0.569
welcomed   0.568

```

```

In [12]: # Bar chart of the 15 most frequent words in the corpus for context.
common = freq.most_common(15)
labels_, counts_ = zip(*common)
plt.figure(figsize=(10, 4))
plt.bar(labels_, counts_, color="darkorange")
plt.title("15 most frequent words in the cleaned corpus")
plt.ylabel("Frequency"); plt.xticks(rotation=45, ha="right"); plt.tight_layout()
plt.show()

```



Observation / Inference:

- The highest-frequency word is a **general high-usage term** (e.g. *said / mr*), which is common in news reporting since articles constantly quote people.
- Even very frequent words receive a **well-defined 100-d embedding**, and their nearest neighbours are other reporting/discourse words - showing the model has placed them sensibly in the vector space.
- Because such words appear in *every* category, their vectors tend to sit near the **centre** of the embedding space and are less discriminative for classification than rarer, topic-specific words.

Task 7 - Domain-specific embeddings for Bioinformatics (3 Marks)

How would you create domain-specific word embeddings for bioinformatics?

1. **Collect a large in-domain corpus** - PubMed/MEDLINE abstracts, PMC full-text articles, clinical notes, gene/protein databases (UniProt, GenBank). General-web text is *not* enough because scientific terminology is under-represented there.
2. **Domain-aware pre-processing & tokenisation** - scientific terms contain hyphens, Greek letters, digits and gene symbols (e.g. *TP53*, *BRCA1*, *5-HT2A*). A normal tokeniser would break these, so we use a biomedical tokeniser (e.g. **scispaCy**) and avoid lower-casing gene symbols.
3. **Train embeddings on the in-domain corpus** - Word2Vec/FastText/GloVe trained *from scratch* on the biomedical corpus, or **fine-tune** a contextual model such as **BioBERT / SciBERT / PubMedBERT** which are BERT models pre-trained on biomedical text.
4. **Sub-word models (FastText)** help a lot because many biological terms share **morphemes** (-ase, -cyte, -genesis), letting the model generalise to rare/unseen terms.

Challenges you might encounter:

- **Highly specialised vocabulary** and a *huge* number of rare terms -> data sparsity.
- **Synonymy & abbreviation ambiguity** - the same entity has many names (*TP53*, *p53*, *tumour protein 53*) and one abbreviation can mean different things (*ER* = oestrogen receptor or endoplasmic reticulum).
- **Polysemy across sub-domains**, multi-word terms ("*epidermal growth factor receptor*"), and constant introduction of **new terms** as research advances.
- **Limited labelled data** and the **cost** of expert annotation.

How to ensure the embeddings capture the unique semantics of biological terms:

- Train on **large, high-quality in-domain corpora** and keep vocabulary that respects scientific tokens.
- Use **sub-word / character n-gram** models (FastText) so morphologically related terms cluster together.
- **Normalise entities** with ontologies/knowledge bases (MeSH, UMLS, Gene Ontology) so synonyms map to the same concept; optionally inject these relations via **retrofitting**.
- Prefer **contextual biomedical models (BioBERT/PubMedBERT)** to resolve abbreviation polysemy.
- **Validate intrinsically** (similarity/analogy on curated biomedical word-pair sets, clustering of known gene families) and **extrinsically** (NER, relation extraction) with domain experts in the loop.

Conclusion

- Loaded and explored the balanced 5-class BBC news dataset, then cleaned the text (lower-casing, punctuation/special-character removal and stopword removal).
- Built **X / y** and trained **CBOW and Skip-Gram** Word2Vec models across **three window sizes**.
- Compared static vs. contextual embeddings, analysed how window size and architecture change a word's nearest neighbours, and inspected the highest-frequency word.
- Discussed how to build and validate **domain-specific embeddings for bioinformatics**.